이음 E:UM

배치 실행·서버 관제·업무 관제를 하나로 잇는 시스템 운영 도구

사건은 하나인데, 도구는 여럿입니다.

- 아침 점검부터 화면을 여러 개 엽니다. 배치는 스케줄러에서, 서버는 서버 관제에서, 업무량은 업무 관제에서 따로 봅니다.
- 서버도, 데이터 저장소도 따로입니다. 데이터는 서로 엮이지 않고 분석을 위해 다루기 어렵습니다.
- 장애가 나면 도구마다 시각을 맞춰 가며 원인을 좇습니다. 어느 코드가 지연을 일으켰는지는 어떤 도구에도 남지 않습니다.

> **이음은 여러 도구를 하나로 합치고, 실행마다 `실행 번호`를 붙여 모든 기록을 잇습니다.**

사용자: 운영 담당자·개발자 대상: 폐쇄망 리눅스 서버

결과물: 하루 만에 만드는 검증판

Q1 그래서 대체 뭘 만드는 건가요?

스케줄러, 서버 관제, 함수 호출 레벨의 업무 관제 도구를 실행 파일 하나로 통합합니다.

2~3쪽

Q2 기존의 뭘 보완하는 건가요?

비어 있는 도구 사이. 도구끼리 이어지지 않고, 데이터는 제품 안에 갇혀 있고, 비용은 도구마다 나갑니다.

4~6쪽

Q3 뭐가 좋아지는 건가요?

시스템의 복잡도와 관리 비용이 줄고, 장애 조사의 근거를 한 흐름에서 확인하고, 쌓인 운영 기록을 직접 분석해 예측의 기반을 만듭니다.

7~12쪽

Q4 어떻게 가능한 건가요?

실행 번호 하나로 여러 도구의 기록을 잇습니다. 검증된 부품을 쓰고 실행 관리와 기록 연결에 구현을 집중해, 실행·연결·활용 세 흐름을 하루에 완성합니다. 수집·저장·알림은 모듈별로 확장할 수 있게 설계합니다.

13~14쪽

Q1 그래서 대체 뭘 만드는 건가요?

도구 여러 개가 하던 일을 실행 파일 하나로.

스케줄러·서버 관제·업무 관제의 추적 수준을 강화하여 하나로 통합합니다.

배치 스케줄러
정해진 시각에 작업 실행
지금: 상용 Control-M

서버 관제
서버·프로세스의 CPU·메모리
지금: 상용 onTune

업무 관제 업무 프로세스 추적
주문 접수·처리·응답시간
+ 함수 호출 단위까지
지금: 사내 APM 함수 단위 없음

→

이음 + 표시가 강화한 추적

- 실행: 플로우·재시도 **+ 띄운 서비스까지** — 번호
- 자원: 서버 **+ 실행별** CPU·메모리 — 번호
- 업무: 접수·처리·응답 **+ 함수 호출** — 번호

통합 개방 DB : 실행 이력 · 서버 자원 · 실행별 자원 · 업무 지표 · 함수 기록

웹 화면
서버·배치판 트리, 대시보드, 타임머신

SQL 조회
읽기 전용

내보내기
CSV·함수 기록

설치 구성

에이전트 1
서버당
실행·수집·함수 기록

+

매니저 1
DB 저장·스케줄링·화면
에이전트를 모아 관리

=

E:UM
실행 파일 하나

같은 실행 파일을 역할만 바꿔 띄웁니다.

운영 요구	이음의 방식
폐쇄망	외부 서비스·CDN 없이 동작
방화벽	에이전트 → 매니저 한 방향. 포트 1개
실행 통제	검토된 작업 정의(YAML)의 명령만 실행. 토큰 인증
서버 부담	함수 추적은 지정한 작업만. 추적 부하도 화면에 표시

한 줄 정의

작업을 띄우는 도구가 지켜보는 일까지 맡습니다.

Q1 그래서 대체 뭘 만드는 건가요? 운영 흐름으로 보기

띄우고, 지켜보고, 돌아보는 일을 번호 하나로 잇습니다.

지금은 단계마다 도구가 바뀌고, 도구끼리 기록이 이어지지 않습니다.

	서비스 준비 작업	서비스	돌아보기
지금 여러 개의 도구	**Control-M** 배치 실행 ✕	**onTune** 서버 자원 **사내 APM** 업무 지표 함수 기록 없음 ✕	**제품별 화면** 따로 조회

✕ 도구가 바뀔 때마다 기록이 끊김 → 사람이 시각·서버 이름으로 맞춤

이음
도구 1개

실행 번호 01JBQ7… 서비스가 끝날 때까지 같은 번호 →

1 띄운다 | 2 지켜본다 | 3 돌아본다

① 띄운다

개장 준비 플로우가 서비스 기동을 책임지고 번호를 발급합니다.

지금은 Control-M

② 지켜본다

시스템 자원 사용 상황, 서비스의 실행 콜스택 정보를 수집하고 추적합니다. 띄운 스크립트가 끝나도 서비스는 같은 번호로 계속 기록됩니다.

지금은 onTune + 사내 APM, 함수 기록은 없음

③ 돌아본다

지난 구간을 다시 펼쳐 함수까지 확인합니다. SQL·CSV로 용량과 배포 전후를 분석합니다.

지금은 제품 화면 안에서만

운영자는 **실행 번호** 하나로 띄운 작업, 지금 상태, 지난 기록을 오갑니다. 도구를 바꿔 가며 시각을 맞추지 않습니다.

Q2 기존의 뭘 보완하는 건가요?

비어 있는 도구 사이를 하나로 잇습니다.

도구마다 제 몫은 합니다. 하지만 도구를 잇는 공통 키가 없고, 어느 코드가 CPU를 썼는지는 어느 도구도 모릅니다.

배치 스케줄러

상용 · Control-M

무엇을 언제 실행했고, 성공했나

단위: 작업

자체 화면 · 자체 저장소

서버 관제

상용 · onTune

어느 서버·프로세스가 CPU·메모리를 썼나

단위: 서버·프로세스

자체 화면 · 자체 저장소

업무 관제

사내 개발 APM

주문이 몇 건 들어와 처리됐고, 응답은 얼마나 걸렸나

단위: 업무 프로세스

자체 화면 · 자체 저장소

공통 키 없음 → 사람이 시각·서버 이름으로 맞춤

함수(코드) 단위 기록 없음 → 필요하면 개발자가 서버에서 직접 측정

데이터는 **제품마다 따로**. 형식·보존 기간·추출 범위는 제품이 정함

비어 있는 곳과 이음이 채우는 것

도구	비어 있는 곳	이음이 채우는 것
배치 스케줄러	띄운 서비스의 이후 상태를 모름. 업무가 바빠도 시각이 되면 실행	**기동부터 종료까지, 서비스의 실행과 자원 기록이 하나로 이어집니다.**
서버 관제	프로세스를 띄운 실행, CPU를 쓴 함수를 모름	**자원 사용이 늘어난 서비스는 CPU를 쓴 함수까지 바로 확인합니다.**
업무 관제	지연 원인이 업무량·자원·코드 중 무엇인지 모름	**업무가 밀리면 업무량·자원·함수가 한 시간축에 펼쳐집니다.**
도구 공통	데이터는 제품 안에. 계약·서버·화면이 도구마다 따로	**통합·개방된 DB와 이를 다루는 실행 파일 하나. 극도의 단순함을 추구합니다.**

보완할 곳

도구 사이의 연결, 코드 단위 기록, 갇힌 데이터.

Q2 기존의 뭘 보완하는 건가요?

빈틈이 만드는 불편

빈틈이 만드는 불편은 세 가지입니다.

불편 1

특이상황 확인이 번거롭다

장중 체결 지연을 지금 따라가는 순서. 어디부터 볼지는 운영자 경험에 달려 있습니다.

1	업무 관제	응답시간 상승, 처리 대기 증가 확인
2	운영 담당	해당 서비스의 서버·프로세스 확인
3	서버 관제	해당 시각 CPU 확인. "CPU를 다 씀"에서 멈춤
4	스케줄러	서비스를 띄운 실행, 같은 시각 다른 작업 확인
5	개발자	폭주는 이미 끝남. **부하를 다시 만들어 재현** 후 측정

여러 도구, 5단계. 원인 코드는 재현부터

불편 2

쌓인 데이터를 쓰기 어렵다

운영 질문 대부분은 두 도구 이상의 데이터를 엮어야 답이 나옵니다.

- 초당 몇 건부터 체결이 밀리나? 증설할 때인가?
  업무 지표 + 자원
- 밀린 구간에 CPU를 쓴 함수는 평소와 뭐가 다른가?
  업무 지표 + 함수 기록(지금은 없음)
- 정산 배치가 끝나는 시각이 매달 늦어지나?
  실행 이력 + 자원
- 배포 뒤 같은 작업의 함수별 CPU 비중이 바뀌었나?
  실행 이력 + 함수 기록

통계·머신러닝도 엮인 원자료가 있어야 시작

불편 3

비용이 많이 든다

매일 쓰는 기능은 일부입니다. 비용은 제품 단위로 나갑니다.

스케줄러 라이선스·유지보수	상용 · 매년
서버 관제 라이선스·유지보수	상용 · 매년
업무 관제(사내 APM) 유지보수	사내 인력
제품별 관리 서버·저장소	도구마다
계약 갱신·교육·문의 창구	제품마다

제품에 없는 기능은 벤더 개발이나 별도 연동을 기다려야 합니다.

쓰는 건 일부, 비용은 도구마다

Q2 기존의 뭘 보완하는 건가요? 이 설계를 고른 이유

필요한 운영 기능과 데이터 연결을 함께 설계합니다.

이음은 실행·수집·저장을 공통 데이터 모델로 구성해, 별도 연동 지점과 운영 구성을 줄이는 방식을 선택했습니다. 보완 방법 네 가지를 같은 기준으로 비교했습니다.

방법	기록을 잇는 방식	추가 연동 개발	관리할 구성 요소	데이터 추출	유지보수 책임
A. 지금 도구 + 연동	도구마다 키가 달라 시각·서버 이름으로 맞춤	도구 사이마다 개발. 제품별 API·로그 형식에 맞춤	제품 3개와 각 관리 서버·저장소, 연동 모듈	제품별 화면·보고서·API. 범위는 제품이 정함	제품은 벤더, 연동 코드는 우리
B. 오픈소스 조합 스케줄러+관제+프로파일러	공통 키와 데이터 모델을 직접 설계	도구 3개 사이 개발	오픈소스 3개, 저장소 3개, 연동 코드	저장소마다 다른 질의 방식	우리. 도구별 업그레이드 대응 포함
C. 통합 관측 제품 상용 APM·관측 플랫폼	관측 데이터는 제품 안에서 연결. 배치 실행은 제품 밖	기존 스케줄러와 연동. 폐쇄망 설치 조건 확인	관측 제품, 기존 스케줄러	벤더 저장소 ·API	벤더. 기능 추가는 벤더 일정
D. 이음	띄울 때 실행 번호를 붙여 처음부터 연결	제품 안에서 처리. 별도 연동 지점 없음	실행 파일 1개 (매니저·에이전트 역할)와 DB	공개 스키마. 읽기 전용 SQL·CSV	우리. 코드 사내 보유

A~C로도 기록을 이을 수는 있습니다. 다만 도구 사이마다 연동 지점이 생기고, 그 연동 코드는 어느 경우든 우리가 관리합니다. 이음은 그 연결을 처음부터 제품 안에 둡니다.

번호는 띄우는 쪽이 붙이면 간단합니다

택배 송장번호처럼, 보낼 때 붙이면 이후 기록이 따라옵니다. 이음이 서비스를 띄우며 번호를 넘기면 자원·업무 지표·함수 기록이 별도 매칭 없이 같은 번호를 답니다.

선택 이유

실행과 수집을 한 데이터 모델로 설계하면 연동 지점이 줄고, 기록은 처음부터 이어집니다.

Q3 뭐가 좋아지는 건가요?

좋아지는 건 세 가지입니다.

장애 대응만 달라지지 않습니다. 평소 운영의 부담과 데이터 활용도 함께 달라집니다.



① 복잡도와 관리 비용이 줄어듭니다

지금 도구마다 계약·관리 서버·화면이 따로

이음 **실행 파일 1개, 화면 1개, 방화벽 포트 1개. 서버 수십 대는 그룹 트리로 한 화면에**

화면 · 대시보드 8쪽, 배치판 9쪽



② 장애 조사의 근거를 한 흐름에서 확인합니다

지금 여러 도구, 5단계. 원인 코드는 재현 후에

이음 **화면 1개, 2단계. 수집된 구간은 재현 없이 CPU를 쓴 함수까지**

화면 · 타임머신 10쪽, 플레임 그래프 11쪽



③ 운영 기록을 직접 분석하고, 예측의 기반을 만듭니다

지금 제품 화면·보고서 안에서만 확인

이음 **운영 데이터 5종을 SQL로 엮어 처리량별 지연, 함수 비중 변화를 분석. 종료 시각 예측·이상 탐지**

확장

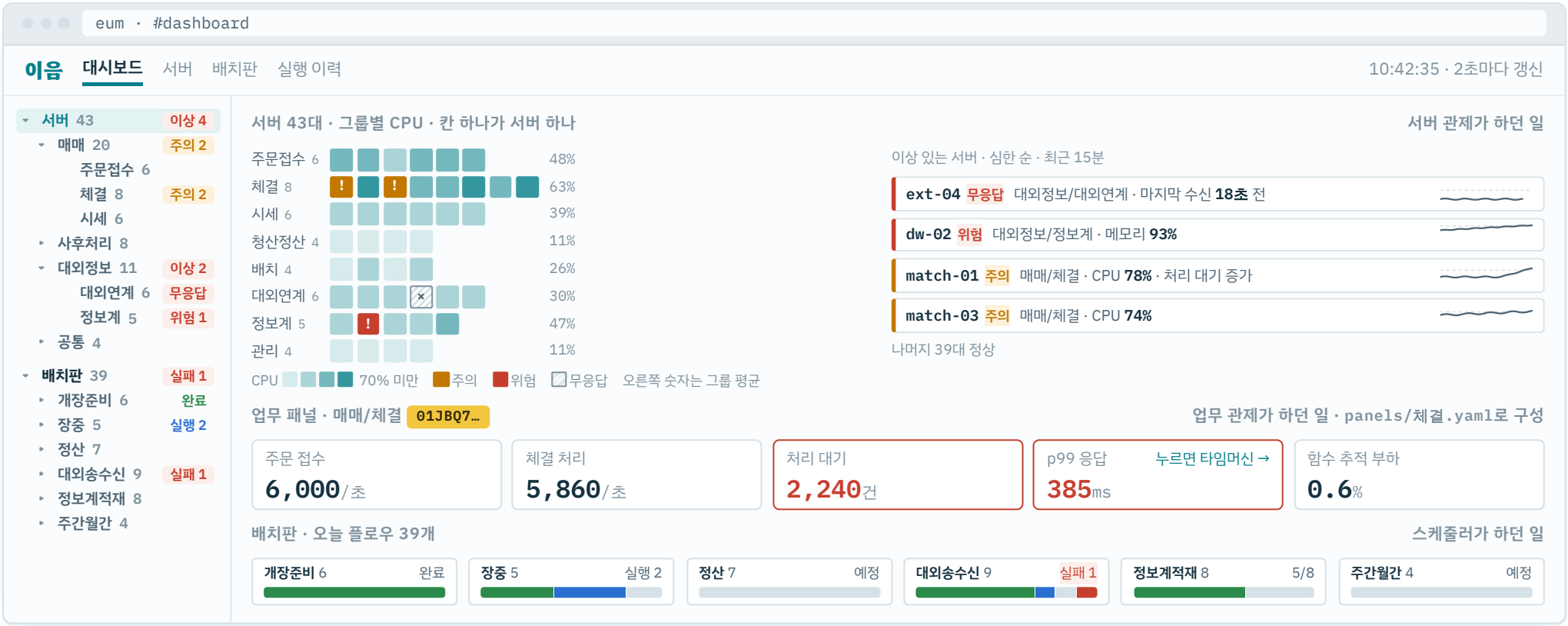
화면 · 데이터 꺼내기 12쪽

Q3 뭐가 좋아지는 건가요?

① 복잡도와 관리 비용

서버·업무·배치를 한 화면에.

서버 수십 대와 플로우 수십 개를 트리로 묶어 봅니다. 지금은 도구를 여러 개 열어야 보이는 내용입니다.



어떻게 쓰나 트리에서 그룹이나 배치판을 고르면 화면이 그 범위로 좁혀집니다. 이상 있는 서버는 목록 맨 위로 올라오고, 업무 패널은 업무마다 정의 파일 하나로 지표를 고릅니다. 업무 타일을 누르면 해당 실행 기록으로 갑니다.

좋아지는 점

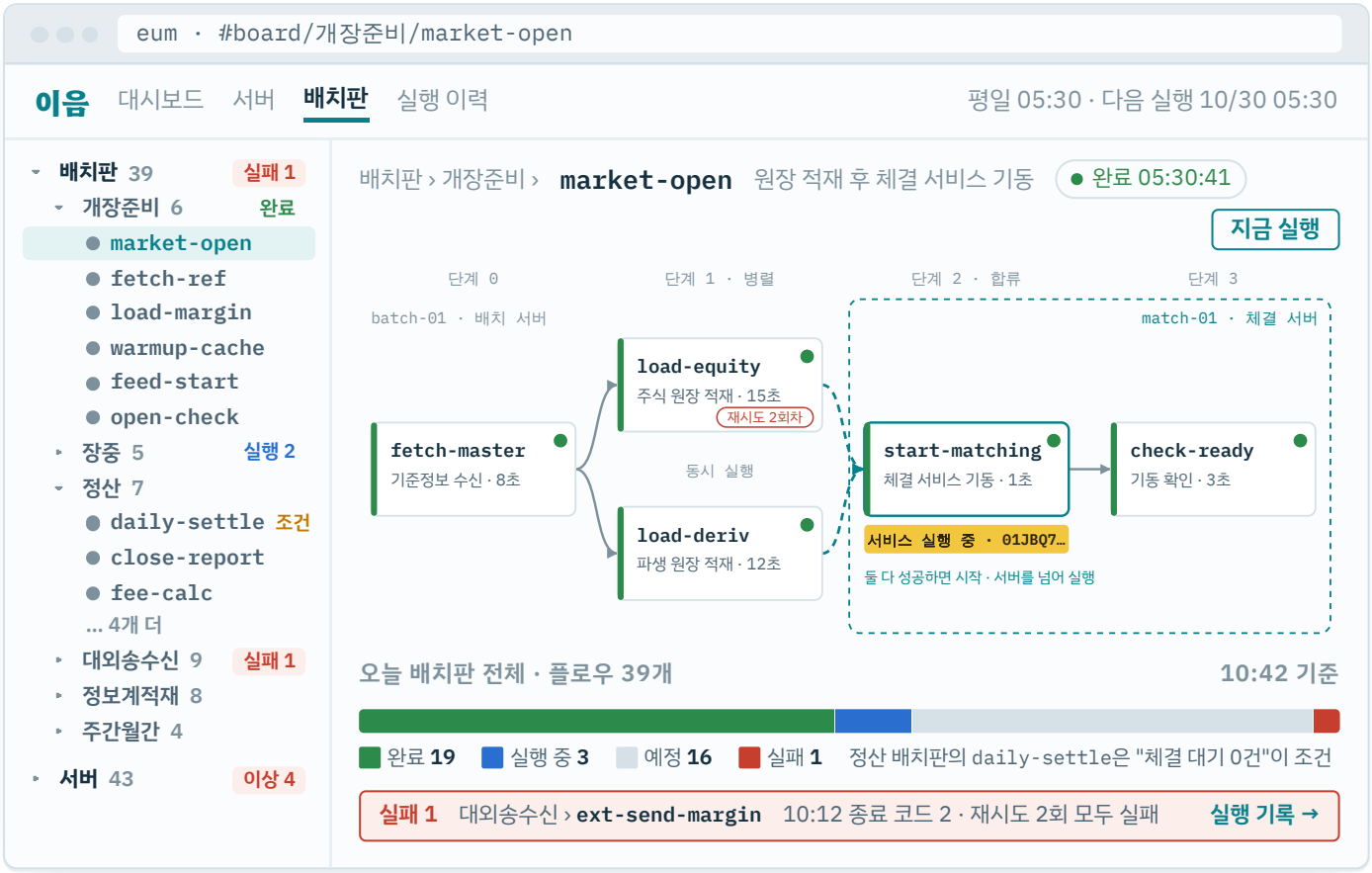
여러 화면을 오가던 점검이 화면 하나로.

Q3 뭐가 좋아지는 건가요?

① 복잡도와 관리 비용 · 배치판

배치 흐름은 그래프로, 실행은 업무에 맞춰.

플로우 수십 개는 배치판 트리로 묶고, 고른 플로우의 병렬·합류·서버 넘기·재시도를 한눈에 봅니다.



설계 시안 · 노드를 누르면 해당 실행 기록(타임머신)으로 이동

무엇을 보나

플로우는 배치판(폴더)으로 묶고, 진행 상황은 트리에 모아 올립니다. 플로우를 고르면 선후행, 병렬과 합류, 서버 간 실행, 재시도가 그래프로 보입니다.

업무를 아는 스케줄러 확장

업무 지표가 같은 DB에 있어 "체결 대기 0건 뒤 정산" 같은 조건을 바로 겁니다. 지표가 끊기면 실행하지 않습니다.

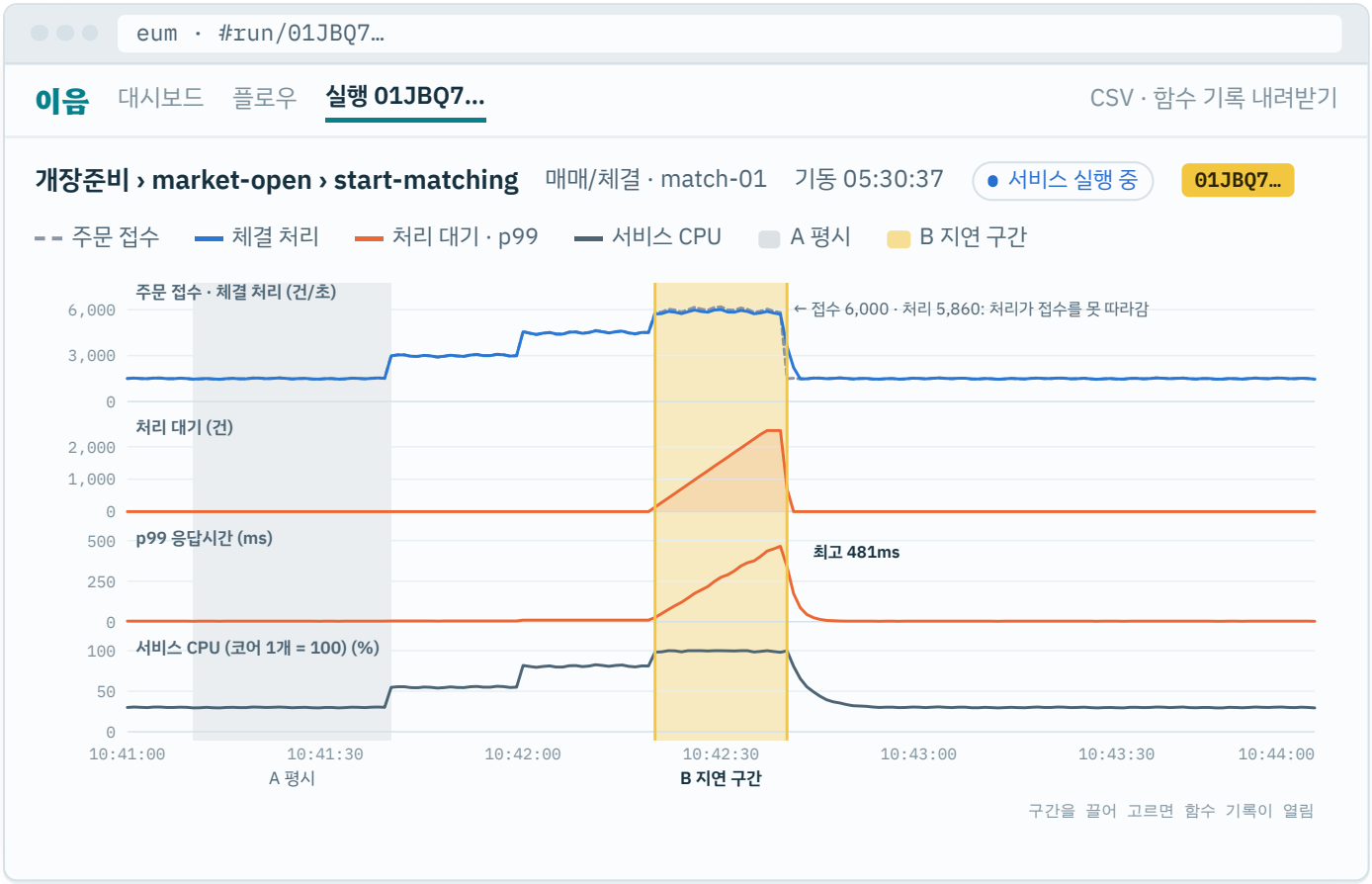
좋아지는 점

시각만 보던 배치가 업무 상황까지 봅니다.

Q3 뭐가 좋아지는 건가요? ② 장애 조사

지나간 장애도 다시 펼쳐 봅니다.

주문이 초당 1,500건에서 6,000건으로 몰린 3분. 서비스의 실행 번호로 모아 한 시간축에 놓았습니다.



설계 시안 · 시연용 C 체결 서비스에 합성 주문으로 만든 부하 예시

무엇을 보나

서비스의 업무 지표, 서비스 CPU, 서버 자원을 한 시간축에. 도구별로 시각을 맞출 필요가 없습니다.

어떻게 쓰나

평시 구간과 지연 구간을 끌어서 고르면 다음 쪽 플레임 그래프가 열립니다.

좋아지는 점

5단계 → 2단계. 타일 클릭, 구간 선택이면 함수까지.

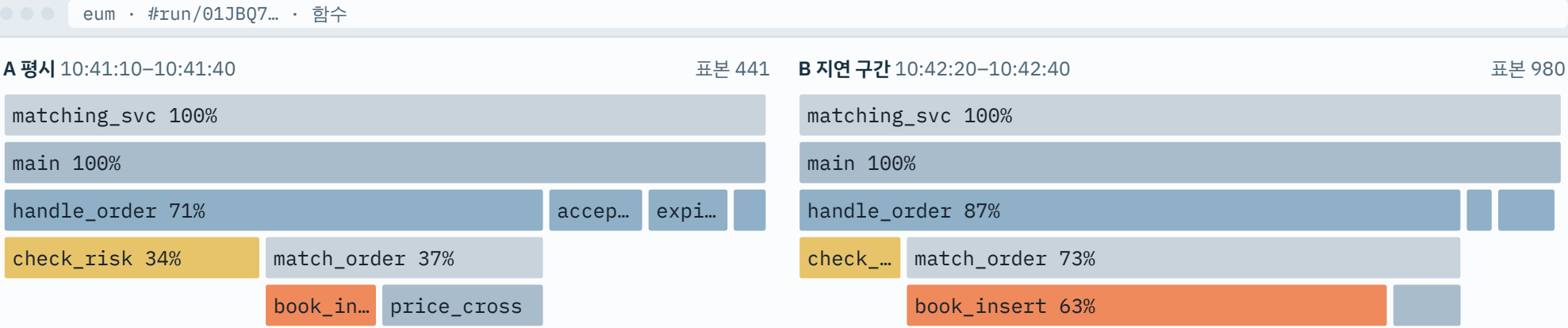
단계 수는 절차 설계 기준. 소요 시간은 같은 시나리오로 실측해 비교합니다.

Q3 뭐가 좋아지는 건가요? ② 장애 조사

CPU를 쓴 함수까지 내려갑니다. 수집된 구간은 재현 없이 확인합니다.

위가 바깥 함수, 아래가 안쪽 함수. 막대 폭은 CPU 사용 비율입니다.

eum · #run/01JBQ7… · 함수



호가 등록 함수 book_insert 비중 **15% → 63%**. 증설보다 **이 코드를 먼저 볼 근거**입니다.

장애마다 보는 기록이 다릅니다

상황	함께 보는 기록	먼저 확인할 것
배치 실패	종료 코드, 로그 끝부분, 선행 작업 결과, 재시도 이력	어느 단계에서 왜 실패했나
서비스 중단	번호로 추적하던 프로세스 수, 직전 자원 변화	언제 사라졌고, 직전에 뭐가 있었나
응답 지연	처리량·대기·응답시간, 자원, 함수별 CPU	업무량·자원·코드 중 어디부터 볼까

범위

함수 기록은 CPU를 쓴 곳만 보여 줍니다. 잠금·디스크 대기는 이번 범위 밖입니다. CPU 비중만으로 원인을 단정하지 않습니다.

좋아지는 점

재현 환경을 만들기 전에 볼 함수가 좁혀집니다.

Q3 뭐가 좋아지는 건가요? ③ 데이터 분석

운영 기록을 직접 분석하고, 예측에 활용할 기반을 만듭니다.

화면은 DB를 보는 방법 중 하나일 뿐입니다. 읽기 전용 SQL과 CSV 내보내기를 처음부터 기능으로 둡니다.

```
-- 최근 5영업일: 초당 몇 건부터 체결이 밀리나
$ eum sql "SELECT in_tps, seconds, avg_p99_ms FROM ..."
 in_tps | seconds | avg_p99_ms
--------+---------+-----------
   1500 | 109,560 |        3.4
   3000 |   6,420 |        4.1
   4500 |   1,140 |        9.8
   6000 |     180 |      254.1

-- 평시 vs 지연 구간, 함수별 CPU 비중(%)
-- 업무 지표와 함수 기록을 실행 번호로 조인
$ eum sql --csv "WITH w AS (...) SELECT ..." > slow.csv
 leaf          | normal_pct | slow_pct
---------------+------------+---------
 book_insert   |       15.2 |     63.4
 check_risk    |       34.0 |     14.0
 price_cross   |       21.8 |      9.6
 expire_orders |       11.1 |      8.2
```

설계 시안 · 두 질의는 같은 스키마의 합성 데이터로 실행 확인 · 수치는 예시

이렇게 씁니다

활용	엮는 데이터	얻는 것
본선 지금 쌓는 데이터로 바로 하는 분석		
용량 판단	업무 지표 + 자원	밀리기 시작하는 처리량 → 증설 시점
코드 개선	업무 지표 + 함수	지연 때 비중이 커지는 함수
배포 전후 비교	실행 이력 + 함수	함수 비중 변화로 성능 저하 발견
확장 쌓인 데이터로 예측		
배치 종료 예측	실행 이력 + 자원	정산 종료 시각 추세
이상 탐지	실행 이력	평소보다 오래 걸린 실행 표시

데이터 주인은 우리

스키마 공개. 컬럼은 추가만 해서 기존 질의가 깨지지 않습니다. 보존 기간·수집 주기도 우리가 정합니다.

좋아지는 점

한 번 보고 끝나던 기록이 분석 재료가 됩니다.

Q4 어떻게 가능한 건가요?

실행 번호 하나가 모든 기록을 잇는 키입니다.

검증된 부품 위에 실행 상태 관리와 기록 연결을 직접 구현합니다.

번호가 이어지는 과정

1. **번호 발급**
   스케줄러가 실행마다 고유 번호를 발급. 재시도는 새 번호.
2. **명령과 함께 전달**
   에이전트가 명령과 번호를 받아 스크립트 실행.
3. **프로세스에 상속**
   환경변수로 넘긴 번호가 스크립트가 띄운 C 서비스까지 따라감. 스크립트가 끝나도 남음.
4. **번호로 찾아 기록**
   에이전트가 번호를 가진 프로세스의 자원·함수를 기록. 서비스는 업무 지표에 번호를 붙여 보냄.

실행 결과	서비스 자원	업무 지표	함수 기록
시작·종료·재시도	CPU·메모리, 1초	접수·처리·대기·p99	10초 단위
01JBQ7…	01JBQ7…	01JBQ7…	01JBQ7…

검증된 기술로 복잡도를 줄임

할 일	쓰는 부품	우리가 만드는 부분
함수 기록	리눅스 커널 내장 perf	대상 프로세스 선택, 결과 압축 저장
저장·조회	SQLite(파일 하나짜리 DB)	공개 스키마, 분석용 뷰, 조회 API
실행 시각	검증된 cron 해석 라이브러리	선후행·재시도 판단, 실행 루프
차트	uPlot(경량 차트)	플로우 그래프, 플레임 그래프
묶어 보기	설정의 그룹 경로, 플로우 폴더	서버·배치판 트리, 그룹별 상태 집계, 업무 패널
배포	Go 단일 실행 파일	매니저·에이전트 역할 분리

구성 · 서버는 에이전트 설정의 그룹 경로로 묶습니다

- **매매/체결** match-01~08
- **매매/주문접수** gw-01~06
- **사후처리/배치** batch-01~04

… 그룹 8개, 서버 43대. 서버마다 에이전트 1개

→

- **매니저** 스케줄러·수집·저장·화면
- 실행 상태 DB + 지표·함수 DB
- 연결은 에이전트 → 매니저 한 방향

Q4 어떻게 가능한 건가요? 하루, 2명

하루에 완성할 흐름은 실행·연결·활용 세 가지입니다.

2명이 AI 코딩 세션 4개를 나눠 맡아 이 세 흐름을 구현합니다. 해커톤 당일 모든 마법이 이뤄집니다.

15:00까지 반드시 완성할 흐름

1 실행

선후행·재시도가 있는 플로우로 서비스를 기동합니다.

화면: 배치판 트리, 플로우 그래프

2 연결

기동 스크립트가 끝나도 업무·자원·함수 기록을 같은 실행으로 조회합니다.

화면: 서버 트리, 대시보드, 타임머신

3 활용

지난 구간을 비교하고, 실제 수집 데이터를 SQL로 꺼냅니다.

화면: 플레임 그래프, SQL·CSV

당일 일정



시간이 남으면 업무 지표 조건 실행(정산 대기), 함수 비교 표, 임계치 알림, 실행 강제 종료

이번에 안 함 로그인·권한, TLS, 이중화, 휴일 캘린더, 자바 함수 추적, 잠금·디스크 대기 분석

미리 확인한 것

함수 이름 복원	운영과 같은 빌드 조건(-02, 프레임 포인터 없음)의 C 시험 프로그램에서 호출 경로 정확히 복원
추적 부담	코어 1개의 **약 0.6%**, 대상 소요 시간 변화는 오차 범위. 시험 프로그램 기준이라 본선에서 다시 재서 화면에 띄움
서비스 추적	스크립트 밖으로 나간 프로세스를 환경변수 표식으로 찾음(일반 사용자 권한)
분석 질의	12쪽 SQL 2개를 합성 데이터로 실행
운영 서버	perf 사용 가능

기능은 끼워 넣는 구조

수집·추적·함수 기록·저장·알림을 인터페이스 6개로 나눕니다. 자바 함수 기록, 시계열 DB, 메신저 알림은 구현만 추가해 붙입니다.

늦어지면 줄이는 순서

서버 열지도(목록으로) → CSV 내보내기 → 구간 끌어 고르기(고정 구간으로) → 플레임 그래프(상위 함수 표로). 세 흐름과 서버·배치판 트리는 끝까지 지킵니다.

본선은 외부망·상용 AI로 개발하고 사내 데이터·소스는 쓰지 않습니다. 시연은 컨테이너 3대와 C 체결 서비스에 모의 서버 40대·플로우 37개를 더해 하며, 서버 자원은 모의 값입니다.

결론 사건은 하나, 도구도 하나.

단절 도구 3개, 저장소도 따로

배치 스케줄러 — 실행 결과

×

서버 관제 — 서버 자원

×

업무 관제 — 업무 지표

도구마다 키가 달라 시각·서버 이름으로 맞춤

실행 번호 →

띄울 때 붙여
처음부터 연결

이음 실행 파일 1개, 통합 개방 DB

실행 결과	01JBQ7…
서버 자원	01JBQ7…
업무 지표	01JBQ7…
함수 콜스택 등 심화 지표	01JBQ7…



① 복잡도·관리 비용

1 · 1 · 1

실행 파일·화면·방화벽 포트

8~9쪽



② 장애 조사

5 → 2단계

수집된 구간은 재현 없이
함수까지

10~11쪽



③ 데이터 분석

5종 → SQL

운영 데이터를 직접 엮어 분석

12쪽

본선

해커톤 당일, 2명이 직접 구현해 개념을 실증합니다.

실행 → 연결 → 활용

띄우고, 지켜보고, 돌아보는 일을 번호 하나로.